

Lab 3: Selection: if-else and switch

Computer Programming () |

Duration: 2 hours (in-lab) | **Topic:** Conditional execution, logical operators, multi-way branching

Objectives

- Use `if`, `if-else`, and `if-else if-else` ladders to express multi-way decisions.
- Combine relational and logical operators (`&&`, `||`, `!`) into conditions.
- Choose between an `if`-ladder and a `switch` statement appropriately.
- Use `switch` fall-through deliberately, and remember the `break`.

Quick Reference

if-else.

```
if (cond) {
    /* runs when cond is non-zero */
} else if (other) {
    /* ... */
} else {
    /* fallback */
}
```

switch.

```
switch (expr) {
    case 1: /* ... */ break;
    case 2:
    case 3: /* shared body */ break;
    default: /* ... */
}
```

Logical operators short-circuit: `a && b` skips `b` if `a` is false; `a || b` skips `b` if `a` is true.

Common pitfall. `=` is assignment; `==` is comparison.

Range tests. You cannot write `0 < x < 10`. Write `x > 0 && x < 10`.

Quick Experiments (Try It)

Try It 1: Assignment inside if – the classic typo

Run this program and observe:

```
#include <stdio.h>
int main(void) {
    int x = 0;
    if (x = 5) { /* assignment, not comparison! */
        printf("true branch\n");
    } else {
        printf("false branch\n");
    }
    printf("x is now: %d\n", x);
    return 0;
}
```

1. Which branch executes? What value does `x` have after the `if`?
2. Change `x = 5` to `x = 0`. Which branch runs now? Why?

3. Change to `x == 5`. Does it behave as you expect?
4. Lesson: assignment = always evaluates to the assigned value; `if (a = 0)` is always false; `if (a = 1)` is always true.

Try It 2: Fall-through in switch – remove one break

Take your switch-based calculator from Exercise 3. Remove the `break` after the `'+'` case only, then test the input `4 + 2`.

1. What is printed? Which cases ran?
2. Now add `break` back and remove it after `'-'`. Test `4 - 2`.
3. What is intentional fall-through useful for? (Hint: look at the take-home trace.)

Try It 3: Range check: the C chain-comparison trap

Predict the output, then run:

```
#include <stdio.h>
int main(void) {
    int x = 50;
    if (0 < x < 10)           /* legal C but NOT what you think */
        printf("in range\n");
    else
        printf("out of range\n");
    return 0;
}
```

1. Did the correct branch run? Explain what `0 < x < 10` actually means to the compiler. (Hint: evaluate `0 < x` first; what type is its result?)
2. Fix the condition with `&&` and verify it now works correctly.

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Character Classification (25 min)

Read a single character and report which category it belongs to: *digit*, *lowercase letter*, *uppercase letter*, *space*, or *other*. Use an if-else if ladder, not `ctype.h`.

In-Lab Exercise 2: Leap Year (25 min)

Read a year and print whether it is a leap year. A year is a leap year if it is divisible by 4 *and* (not divisible by 100 *or* divisible by 400). Express this in a single boolean expression using `&&` and `||`.

In-Lab Exercise 3: Simple Calculator with switch (35 min)

Read two doubles and a single-character operator (`+`, `-`, `*`, `/`) and print the result. Use a `switch` on the operator. Handle division by zero gracefully (print an error message instead of dividing).

In-Lab Exercise 4: Triangle Type (35 min)

Read three positive numbers as side lengths. First, check the triangle inequality: each side must be shorter than the sum of the other two. If invalid, print **Not a triangle**. Otherwise, classify as *Equilateral*, *Isosceles*, or *Scalene*.

AI Collaboration

Try these prompts with an AI tool:

- “Explain the dangling-else problem in C with a concrete example and show how braces fix it.”
- “When should I use `switch` instead of an `if-else` if ladder? What are the restrictions on `switch` expressions in C?”
- “Why does `if (x = 5)` always enter the true branch? What compiler warning catches this?”

Critical check – verify, don’t just accept: Ask the AI to write a grade classifier using `switch` (A/B/C/D/F from a score). Before running, count how many `break` statements appear and whether every case has one. Compile and test with edge inputs: exactly 90, exactly 100, 0, and -1 . Record any behavior that differs from the AI’s stated output.

Know the limit: AI tools regularly omit or misplace `break` statements in `switch` blocks and rarely distinguish intentional fall-through from accidental fall-through – inspect every case boundary yourself.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Dangling else

Trace and predict for `x = 0`:

```
#include <stdio.h>
int main(void) {
    int x = 0;
    if (x > 0)
        if (x > 10) printf("big\n");
    else printf("small\n");
    return 0;
}
```

Hint: an `else` associates with the nearest unmatched `if`.

Trace & Predict 2: switch fall-through

Predict the output for inputs 1, 2, and 3:

```
#include <stdio.h>
int main(void) {
    int n; scanf("%d", &n);
    switch (n) {
        case 1: printf("one ");
        case 2: printf("two "); break;
    }
```

```
        case 3: printf("three ");
        default: printf("end");
    }
    printf("\n");
    return 0;
}
```

Find the Bug 1: Assignment vs. comparison + missing break

Find the bugs and rewrite the corrected program:

```
#include <stdio.h>
int main(void) {
    int grade;
    scanf("%d", &grade);
    if (grade = 100) {
        printf("Perfect!\n");
    }
    switch (grade / 10) {
        case 10:
        case 9: printf("A\n");
        case 8: printf("B\n"); break;
        default: printf("C or below\n");
    }
    return 0;
}
```

Write Code on Paper 1: Maximum of three integers

Write a complete program that reads three integers and prints the largest. Do not use any library function such as `fmax`. Use nested `if-else`.

Write Code on Paper 2: BMI category

Write a complete program that reads weight in kilograms and height in meters, computes BMI (w/h^2), and prints one of the four categories: *Underweight* (< 18.5), *Normal* ($[18.5, 25)$), *Overweight* ($[25, 30)$), *Obese* (≥ 30). Print the BMI value with one decimal.